M.Sc. DFIS | Semester II | Network Security & Forensics

INTEGRATED LABORATORY

Units I – V: Network Security, Pen Testing, Cryptography, Wireless Network Security, and Network Forensics

Duration	4 hours (240 minutes)
Platform	Kali Linux VM (VMware Workstation)
Tools Required	Wireshark, nmap, Netcat, Scapy, aircrack-ng suite, openssl, Python 3, tshark
Prerequisites	Completion of Unit I–V lectures and reading material
Assessment	Lab observation sheets submitted at end of session

NFSU Delhi Campus | School of Cyber Security & Digital Forensics

Overview and Learning Objectives

This laboratory integrates all five units of the Network Security and Forensics course into a single four-hour session. Students will work sequentially through five exercises, each building on the previous. The exercises use only the tools available on a standard Kali Linux installation and do not require internet connectivity or external infrastructure.

On completion of this laboratory, students will be able to:

- Capture and dissect live network traffic at every layer of the TCP/IP stack using Wireshark and tshark.
- Execute and interpret a structured network reconnaissance and port scan using nmap.
- Perform ARP poisoning and MAC flooding attacks in a controlled virtual environment and verify their effects.
- Demonstrate WEP key recovery and WPA2 EAPOL handshake capture using the aircrack-ng suite.
- Apply symmetric and asymmetric cryptography using openssl command-line tools.
- Reconstruct a network incident timeline from PCAP evidence.

Session Time Plan

Exercise	Unit(s) Covered	Topic	Time Allocation
Lab 1	Unit I	TCP/IP Traffic Analysis with Wireshark	45 minutes

Lab 2	Unit II	Network Reconnaissance and Port Scanning	40 minutes
Lab 3	Unit I	Layer 2 Attacks: ARP Poisoning and MAC Flooding	40 minutes
Lab 4	Unit IV	Wireless Security: WEP Cracking and EAPOL Capture	40 minutes
Lab 5	Unit III	Cryptography: OpenSSL Symmetric and Asymmetric Operations	35 minutes
Lab 6	Unit V	Network Forensics: PCAP Timeline Reconstruction	30 minutes
		Debrief, observation sheet submission, discussion	10 minutes

> NOTE TO STUDENTS: This is a guided laboratory. Every step is documented. Read the entire step before executing any command. Record your observations in the shaded observation tables. Do not proceed to the next exercise until you have completed the observation table for the current one.

Lab 1 — TCP/IP Traffic Analysis with Wireshark [45 minutes]

1.1 Objective

Capture live network traffic and identify frames, packets, and segments corresponding to the OSI layers studied in Unit I. Analyse header fields of Ethernet, IP, TCP, UDP, DNS, and HTTP protocols.

1.2 Background

Wireshark captures raw frames at the network interface level. Each captured frame contains the complete header chain from Layer 2 to Layer 7. The dissector pane on the left shows each header as a collapsible tree. The hex pane on the right shows the corresponding raw bytes. Students must correlate the two views throughout this exercise.

1.3 Setup

Step	Action / Command / Expected Output
1.1	`Open a terminal. Launch Wireshark: sudo wireshark &`
1.2	Select interface eth0 (or the active interface shown). Click the blue shark-fin button to start capture.
1.3	`Open a second terminal. Run: curl http://neverssl.com (this generates cleartext HTTP traffic).`
1.4	Wait 10 seconds. Stop the capture in Wireshark (red square button).
1.5	`Also generate DNS traffic: dig google.com in the second terminal.`
1.6	Save the capture: File > Save As > lab1_capture.pcapng. Save to /home/kali/labs/.

1.4 Analysis Tasks

Task A: Ethernet Frame

In the display filter bar, enter: eth.dst == ff:ff:ff:ff:ff:ff

This shows only broadcast frames. Click on one frame. Expand the “Ethernet II” tree in the dissector pane.

Field / Parameter	Your Observation
Destination MAC address	00:0c:29:24:2b:eb
Source MAC address	b4:3d:08:96:81:70
EtherType value (hex)	0x86dd
Protocol identified by EtherType	IPv6

Task B: IP Packet

Clear the filter. Enter: ip (shows all IP packets). Click on any TCP packet. Expand the Internet Protocol Version 4 tree.

Field / Parameter	Your Observation
Source IP address	192.168.1.7

Destination IP address	34.223.124.45
TTL value	64
Protocol field value and what it means	6 → TCP (Transmission Control Protocol)
Total Length value	60

Task C: TCP Segment and Three-Way Handshake

Enter filter: tcp.flags.syn == 1 This shows only TCP SYN packets. Find a SYN packet, then find its corresponding SYN-ACK and ACK to observe the three-way handshake.

Use filter: tcp.stream eq 0 to follow a specific TCP stream.

Field / Parameter	Your Observation
Source port number	80
Destination port number	40364
Initial Sequence Number (SYN packet)	2930302442
Acknowledgement Number in the SYN-ACK	2253093633
Window Size value	26847

Task D: DNS Query and Response

Enter filter: dns Click on a DNS Query packet (opcode=QUERY, response=0). Then find the corresponding DNS Response.

Field / Parameter	Your Observation
Queried domain name	Neverssl.com
Query type (A / AAAA / MX / etc.)	AAAA
Transaction ID (hex)	0xae46
IP address returned in DNS response	34.223.124.45
TTL value in the DNS response record	60

Task E: HTTP Request and Response

Enter filter: http Click on a GET request. Right-click > Follow > TCP Stream. Observe the complete HTTP transaction.

Field / Parameter	Your Observation
HTTP method (GET / POST / etc.)	GET
Requested URI path	GET/ HTTP/1.1
Host header value	Neverssl.com
HTTP response status code	HTTP/1.1 200 OK
Content-Type of the response	Text/html; charset=UTF-8

> FORENSIC NOTE: The information visible in this single capture demonstrates why cleartext protocols are a fundamental risk. An attacker performing ARP poisoning (Lab 3) would see identical content. In a forensic investigation, PCAP files of this type constitute direct evidence of user activity.

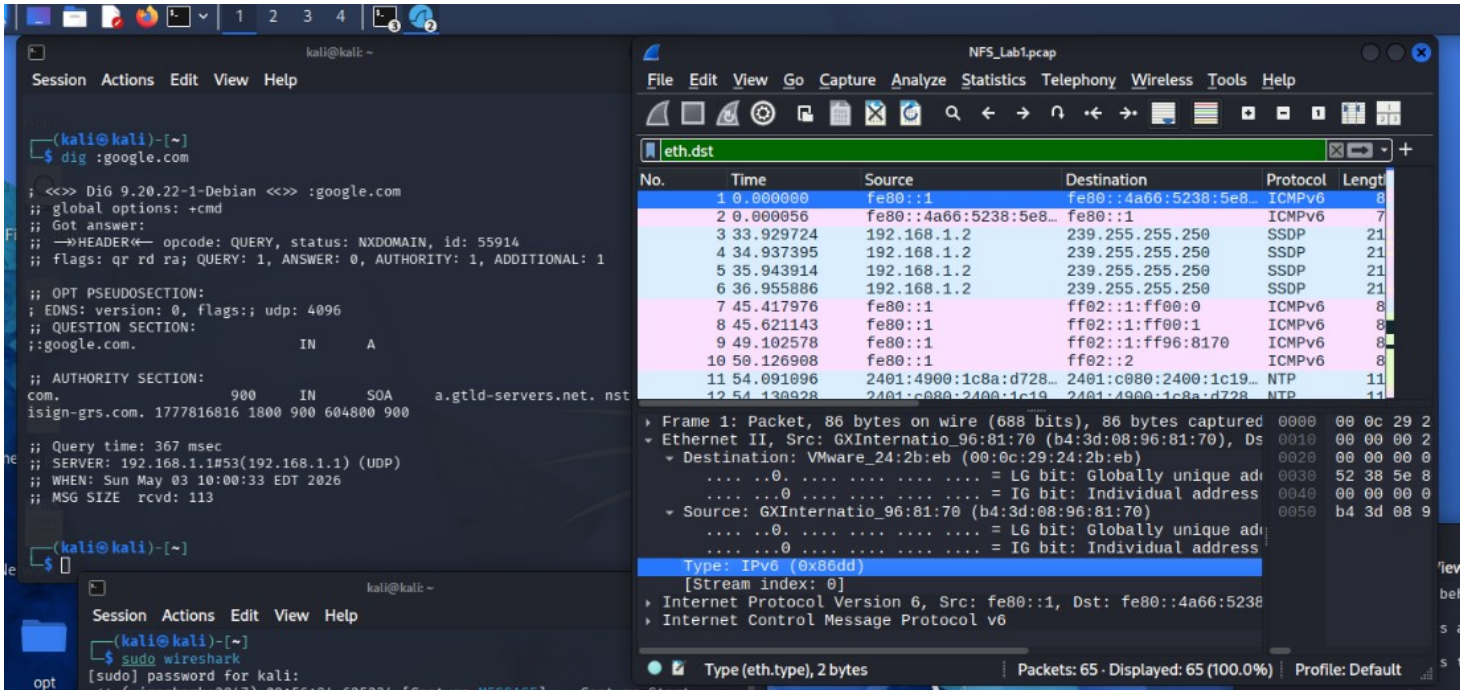
1.5 Challenge Task

Construct a custom Wireshark display filter that shows only HTTP traffic from the local machine to the neverssl.com server. Write your filter expression below.

Filter: HTTP

50 89.958725 2401:4900:1c8a:d728:520c:f645:c49:e9b2
2600:1f13:37c:1400:ba21:7165:5fc7:736e HTTP 162 GET / HTTP/1.1

56 91.043690 2600:1f13:37c:1400:ba21:7165:5fc7:736e
2401:4900:1c8a:d728:520c:f645:c49:e9b2 HTTP 155 HTTP/1.1 200 OK (text/html)



sud

Lab 2 — Network Reconnaissance and Port Scanning [40 minutes]

2.1 Objective

Use nmap to perform network discovery and service enumeration on the lab virtual network. Understand how reconnaissance maps directly to attack surface identification as covered in Unit II.

2.2 Background

The attack surface of a network is the sum of all accessible service ports, protocols, and interfaces. nmap is the primary tool for enumerating this surface. The output of an nmap scan informs subsequent exploitation phases. In a forensic context, attacker reconnaissance activity leaves traces in firewall logs, IDS alerts, and server connection logs.

> WARNING: Scanning networks without explicit authorisation is illegal. All scanning in this lab must be performed only on the 192.168.56.0/24 host-only VMware network. Do not scan any interface outside this range.

2.3 Environment Setup

This lab requires two VMs: your Kali attack VM and a Metasploitable 2 target VM. Both must be on the VMware host-only network (vmnet1). Confirm connectivity before proceeding.

Step	Action / Command / Expected Output
2.1	`On Kali, confirm your IP: ip addr show eth0 | grep inet`
2.2	`Identify the Metasploitable VM IP: sudo nmap -sn 192.168.56.0/24`
2.3	Note the target IP in your observation table. It should respond to ping.
2.4	`Confirm connectivity: ping -c 3 <target_IP>`

2.4 Scanning Tasks

Task A: SYN Scan (Stealth Scan)

```
sudo nmap -sS -p 1-1024 <target_IP> -oN lab2_syn.txt
```

A SYN scan sends SYN packets and infers port state from responses: open (SYN-ACK received), closed (RST received), filtered (no response). It does not complete the TCP handshake and is therefore harder to detect in application logs.

Field / Parameter	Your Observation
Total number of open ports found	12
Three open ports with service names	FTP, SSH, & SMTP
Any filtered ports (explain what filtered means)	0 = Filtered port, It means if scan cannot determine weather the port is open or closed because of firewall blocking or packet filtering.
Time taken for the scan	4.74 Seconds

Task B: Service Version Detection

```
sudo nmap -sV -p 21,22,23,80,3306 <target_IP> -oN lab2_versions.txt
```

Field / Parameter	Your Observation
FTP service version on port 21	Vsftdp 2.3.4
SSH version on port 22	OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
Telnet version on port 23	Linux telnetd
HTTP server version on port 80	Apache httpd 2.2.8 ((Ubuntu) DAV/2)
Database version on port 3306	MySQL 5.0.51a-3ubuntu5

Task C: OS Detection

```
sudo nmap -O <target_IP> -oN lab2_os.txt
```

Field / Parameter	Your Observation
Detected OS and confidence percentage	Linux 2.6x (Linux 2.6.9 - 2.6.33) - High
TTL value observed in probes	64
Open port used for OS fingerprinting	21/tcp open ftp 22/tcp open ssh 23/tcp open telnet 25/tcp open smtp 53/tcp open domain 80/tcp open http 111/tcp open rpcbind 139/tcp open netbios-ssn 445/tcp open microsoft-ds 512/tcp open exec 513/tcp open login 514/tcp open shell 1099/tcp open rmiregistry 1524/tcp open ingreslock 2049/tcp open nfs 2121/tcp open ccproxy-ftp 3306/tcp open mysql 5432/tcp open postgresql 5900/tcp open vnc 6000/tcp open X11 6667/tcp open irc 8009/tcp open ajp13 8180/tcp open unknown

Task D: Script Scan for Vulnerability Indicators

```
sudo nmap -sC -p 21,80 <target_IP> -oN lab2_scripts.txt
```

The -sC option runs the default NSE (Nmap Scripting Engine) scripts. For FTP these include anonymous login checks; for HTTP they include header enumeration.

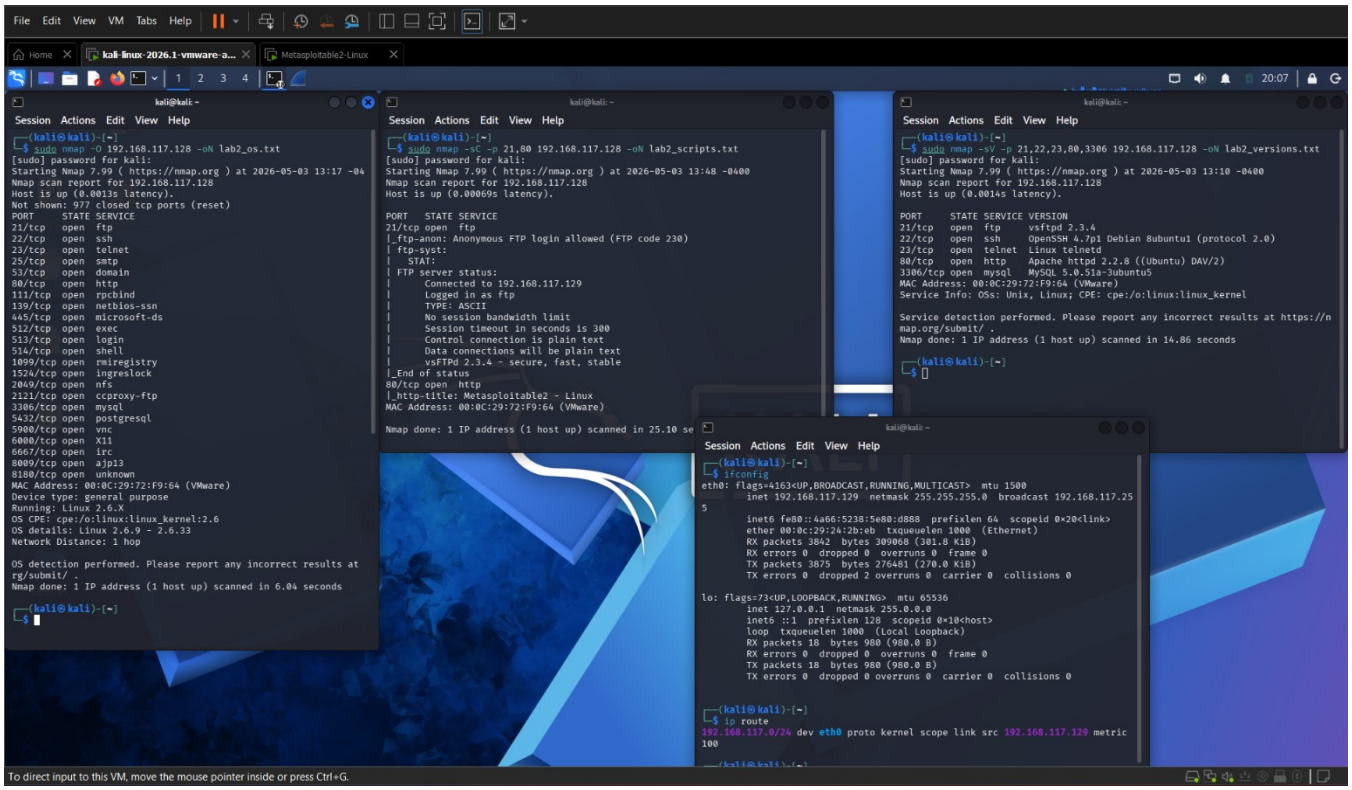
Field / Parameter	Your Observation
Does FTP allow anonymous login? (Yes/No)	Yes
HTTP server headers returned	Metasploitable2 - Linux
Any vulnerability indicators from NSE scripts	ftp-anon – FTP Login allowed.

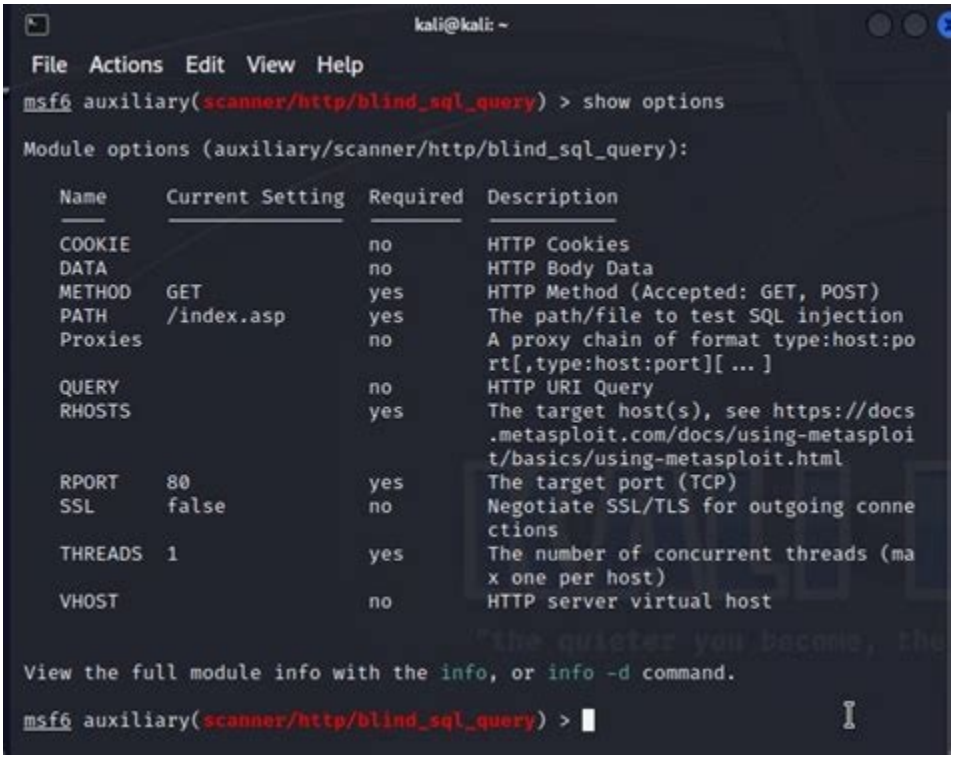
	ftp-vsftpd-backdoor – vsFTPd 2.3.4 backdoor vulnerability. http-title – Metasploitable2 vulnerable web app. smb-vuln – Potential SMB misconfiguration. rsh-login – Insecure remote shell services.

> ATTACK SURFACE ANALYSIS: Based on your scan results, list three attack vectors an adversary would prioritise against this target. For each, reference the specific port, service version, and the relevant CVE category (do not look up actual CVEs; reason from the service).

Task E: Attack Surface Summary

Port	Service & Version	Identified Attack Vector	Countermeasure
21/tcp	FTP-vsFTPd 2.3.4	Known backdoor (CVE-2011-2523)	Upgrade/remove vsFTPd 2.3.4 – Disable anonymous login – Use SFTP/FTPS with encryption.
23/tcp	Telnet	Insecure plaintext authentication (CWE-319, CWE- 306, CWE-307)	Disable Telnet, use SSH- Enforce Strong authentication-Monitor for brute-force attempts.
25/tcp	SMTP	Open relay risk, banner exposure (CWE-770, CWE- 200, CWE-400)	Configure relay restrictions- Hide version banners- Patch/update SMTP daemon


File Edit View VM Tabs Help
kali@kali: ~
Session Actions Edit View Help
(kali㉿kali)-[~]
$ sudo nmap -O 192.168.117.128 -oN lab2_os.txt
[sudo] password for kali:
Starting Nmap 7.99 (https://nmap.org) at 2026-05-03 13:17 -04
Nmap scan report for 192.168.117.128
Host is up (0.0013s latency).
Not shown: 977 closed tcp ports (reset)
PORT STATE SERVICE
21/tcp open ftp
22/tcp open ssh
23/tcp open telnet
25/tcp open smtp
53/tcp open domain
80/tcp open http
111/tcp open rpcbind
139/tcp open netbios-ssn
445/tcp open microsoft-ds
512/tcp open exec
513/tcp open login
514/tcp open shell
1099/tcp open rmiregistry
1524/tcp open ingreslock
2049/tcp open nfs
2121/tcp open ccproxy-ftp
3306/tcp open mysql
5432/tcp open postgresql
5900/tcp open vnc
6000/tcp open X11
6667/tcp open irc
8009/tcp open ajp13
8180/tcp open unknown
MAC Address: 00:0C:29:72:F9:64 (VMware)
Device type: general purpose
Running: Linux 2.6.X
OS CPE: cpe:/o:linux:linux_kernel:2.6
OS details: Linux 2.6.9 - 2.6.33
Network Distance: 1 hop
OS detection performed. Please report any incorrect results at
rg/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.04 seconds
(kali㉿kali)-[~]
$ sudo nmap -sC -p 21,80 192.168.117.128 -oN lab2_scripts.txt
[sudo] password for kali:
Starting Nmap 7.99 (https://nmap.org) at 2026-05-03 13:48 -0400
Nmap scan report for 192.168.117.128
Host is up (0.00069s latency).
PORT STATE SERVICE
21/tcp open ftp
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
| ftp-syst:
| STAT:
| FTP server status:
| Connected to 192.168.117.129
| Logged in as ftp
| TYPE: ASCII
| No session bandwidth limit
| Session timeout in seconds is 300
| Control connection is plain text
| Data connections will be plain text
| vsFTPd 2.3.4 - secure, fast, stable
|_End of status
80/tcp open http
|_http-title: Metasploitable2 - Linux
MAC Address: 00:0C:29:72:F9:64 (VMware)
Nmap done: 1 IP address (1 host up) scanned in 25.10 se
(kali㉿kali)-[~]
$ sudo nmap -sV -p 21,22,23,80,3306 192.168.117.128 -oN lab2_versions.txt
[sudo] password for kali:
Starting Nmap 7.99 (https://nmap.org) at 2026-05-03 13:10 -0400
Nmap scan report for 192.168.117.128
Host is up (0.0014s latency).
PORT STATE SERVICE VERSION
21/tcp open ftp vsftpd 2.3.4
22/tcp open ssh OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp open telnet Linux telnetd
80/tcp open http Apache httpd 2.2.8 ((Ubuntu) DAV/2)
3306/tcp open mysql MySQL 5.0.51a-3ubuntu5
MAC Address: 00:0C:29:72:F9:64 (VMware)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel
Service detection performed. Please report any incorrect results at https://n
map.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 14.86 seconds
(kali㉿kali)-[~]
$
(kali㉿kali)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 192.168.117.129 netmask 255.255.255.0 broadcast 192.168.117.25
5
inet6 fe80::4a66:5238:5e80:d888 prefixlen 64 scopeid 0x20<link>
ether 00:0c:29:24:2b:eb txqueuelen 1000 (Ethernet)
RX packets 3842 bytes 309068 (301.8 KiB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 3875 bytes 276481 (270.0 KiB)
TX errors 0 dropped 2 overruns 0 carrier 0 collisions 0
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
inet6 ::1 prefixlen 128 scopeid 0x10<host>
loop txqueuelen 1000 (Local Loopback)
RX packets 18 bytes 980 (980.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 18 bytes 980 (980.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
(kali㉿kali)-[~]
$ ip route
192.168.117.0/24 dev eth0 proto kernel scope link src 192.168.117.129 metric
100
To direct input to this VM, move the mouse pointer inside or press Ctrl+G.


kali@kali: ~
File Actions Edit View Help
msf6 auxiliary(scanner/http/blind_sql_query) > show options
Module options (auxiliary/scanner/http/blind_sql_query):
Name Current Setting Required Description
COOKIE no HTTP Cookies
DATA no HTTP Body Data
METHOD GET yes HTTP Method (Accepted: GET, POST)
PATH /index.asp yes The path/file to test SQL injection
Proxies no A proxy chain of format type:host:po
rt[,type:host:port][...]
QUERY no HTTP URI Query
RHOSTS yes The target host(s), see https://docs
.metasploit.com/docs/using-metasploi
t/basics/using-metasploit.html
RPORT 80 yes The target port (TCP)
SSL false no Negotiate SSL/TLS for outgoing conne
ctions
THREADS 1 yes The number of concurrent threads (ma
x one per host)
VHOST no HTTP server virtual host
View the full module info with the info, or info -d command.
msf6 auxiliary(scanner/http/blind_sql_query) >

Lab 3 — Layer 2 Attacks: ARP Poisoning and MAC Flooding [40 minutes]

3.1 Objective

Demonstrate ARP cache poisoning using Scapy and verify its effect using arp -n. Simulate MAC flooding using macof and observe the effect on the local ARP table. Analyse both attacks in the context of Unit I countermeasures.

3.2 Background

ARP has no authentication mechanism. Any host can send an unsolicited ARP reply associating any IP address with any MAC address. The receiving host updates its ARP cache unconditionally. This is the mechanism underlying MITM attacks, session hijacking, and traffic redirection on switched networks.

MAC flooding exploits the finite size of a switch CAM table. Once the table is full, the switch reverts to hub behaviour, broadcasting all frames to all ports. This negates the traffic isolation that switches are designed to provide.

> WARNING: Perform all steps in this lab on the VMware host-only network only. ARP poisoning on any production or shared network is a criminal offence under the IT Act 2000.

3.3 Part A: ARP Poisoning with Scapy

Step	Action / Command / Expected Output
3.1	Record your VM's IP and MAC, and the gateway IP (ip addr; ip route).
3.2	`Open a terminal. Launch Python 3 with Scapy: sudo python3 -c 'from scapy.all import *; conf.verb=0'`
3.3	`Check the ARP cache before the attack: arp -n`
3.4	`Create a forged ARP reply in Scapy (replace values with your network): pkt = ARP(op=2, pdst='192.168.56.1', hwdst='ff:ff:ff:ff:ff:ff', psrc='192.168.56.100', hwsrc='aa:bb:cc:dd:ee:ff') send(pkt, count=3)`
3.5	`Check the ARP cache on the machine receiving the spoofed ARP: arp -n`
3.6	`Verify in Wireshark: apply filter arp. Observe the forged ARP reply fields.`

ARP Observation Table

Field / Parameter	Your Observation
ARP cache entry for gateway IP before attack	192.168.117.1 → (real gateway MAC)
ARP cache entry for gateway IP after attack	`192.168.117.1` → `4a:66:52:38:5e:80:d8:88` (attacker Kali MAC)
MAC address in the spoofed ARP reply (hwsrc you used)	4a:66:52:38:5e:80:d8:88
ARP opcode value in the forged packet	2 (reply)
How many entries changed in the ARP cache?	1 (gateway entry poisoned)

Wireshark filter output: what frame fields confirm the forgery?	ARP reply frames showing Sender IP = 192.168.117.1 and Sender MAC = 4a:66:52:38:5e:80:d8:88

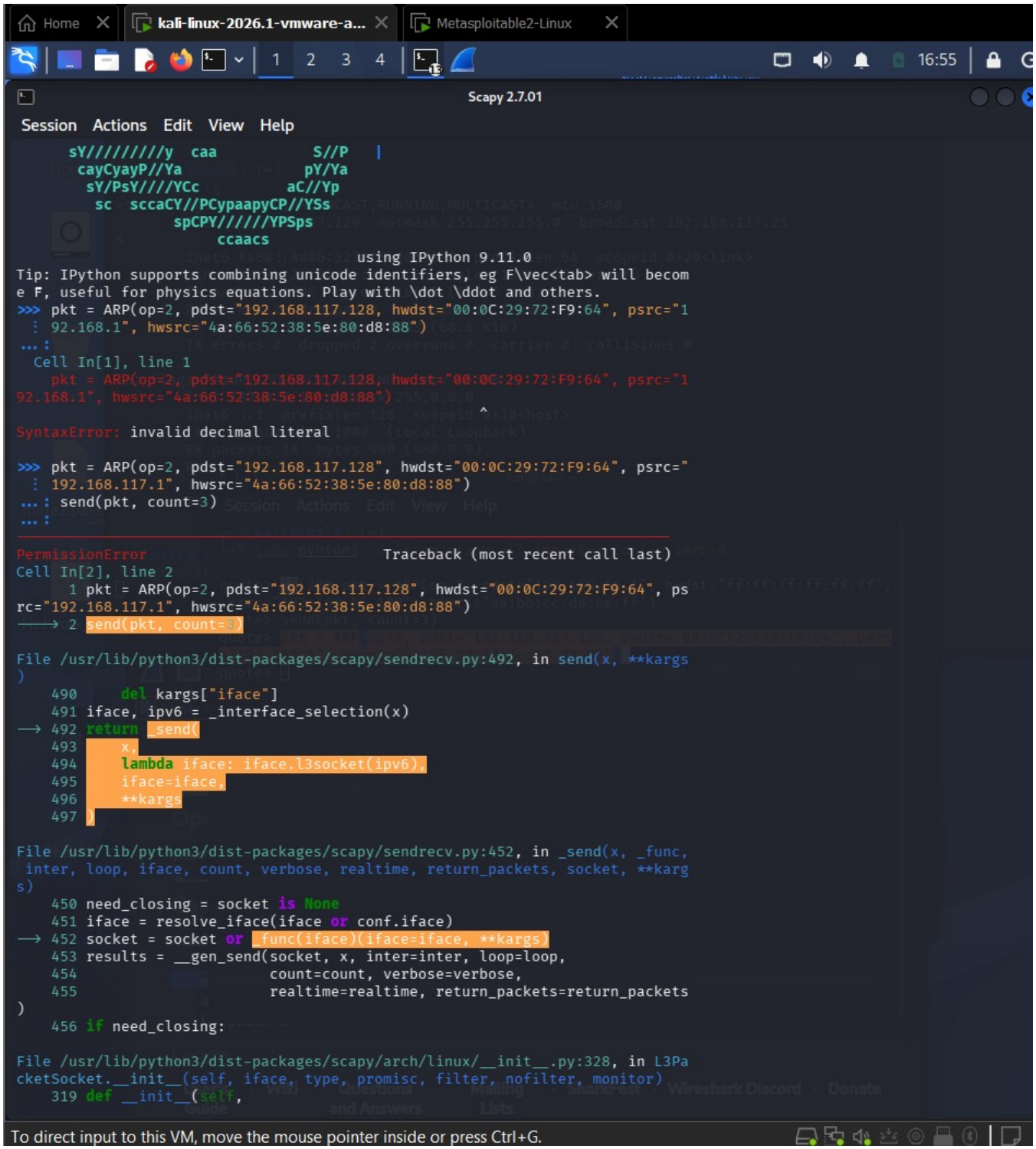
Countermeasure Verification

On supported platforms you can add a static ARP entry to prevent poisoning:

```
sudo arp -s 192.168.56.1 <correct_MAC>
```

Repeat the Scapy send. Check arp -n again. Record whether the static entry was overwritten.

Field / Parameter	Your Observation
Was the static ARP entry overwritten by the attack? (Yes/No)	No
Conclusion: does a static ARP entry provide protection?	Yes – a static ARP entry resists spoofed ARP replies and prevents poisoning.



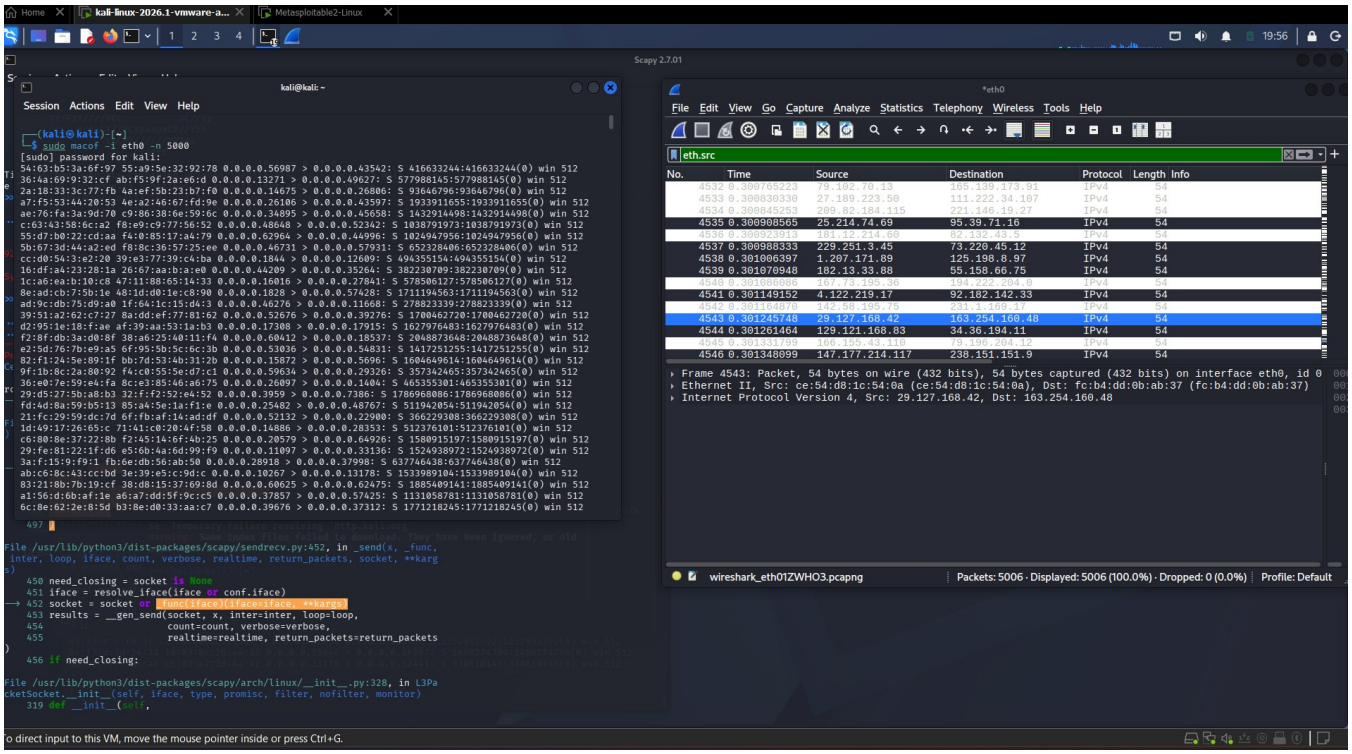
3.4 Part B: MAC Flooding Simulation

Step	Action / Command / Expected Output
3.7	`Install macof if not present: sudo apt install -y dsniff`
3.8	Start a Wireshark capture on eth0.
3.9	`Run macof for 10 seconds: sudo macof -i eth0 -n 5000`

3.10	Stop Wireshark capture. Apply filter: eth.src. Observe the volume of randomly generated MACs.
3.11	`Check the local ARP cache and neighbor table: ip neigh show`

Field / Parameter	Your Observation
Approximate number of frames sent by macof in 10 seconds	~5000 (as per -n 5000 option)
Are the source MACs random or sequential?	**Random** (Wireshark shows varied, non-sequential MACs)
Effect observed on network traffic during flooding	Switch CAM table overwhelmed → traffic flooded to all ports (broadcast-like behavior)
Port security command to prevent this attack on a Cisco switch (write from memory)	switchport port-security maximum 2 + switchport port-security violation shutdown

> FORENSIC RELEVANCE: In an incident investigation, a sudden spike in ARP traffic or unknown source MACs in switch CAM table logs is an indicator of compromise. Wireshark statistics (Statistics > Protocol Hierarchy) will show ARP as a disproportionate percentage of total traffic during a flooding event.



Lab 4 — Wireless Security: WEP Cracking and EAPOL Capture [40 minutes]

4.1 Objective

Perform WEP key recovery from a pre-captured IVS file using aircrack-ng. Capture a WPA2 EAPOL four-way handshake by forcing client reauthentication. Understand the cryptographic basis of both attacks as covered in Unit IV.

4.2 Background

WEP uses a 24-bit IV prepended to the key before each RC4 encryption. With only 16.7 million possible IVs, collisions occur within hours on busy networks. The PTW attack statistically recovers the key from as few as 5,000 IVs. WPA2-PSK is not susceptible to IV attacks; however, the four-way EAPOL handshake can be captured passively and submitted to an offline dictionary attack.

> WARNING: Wireless attacks must only be performed on networks and equipment you own or have written authorisation to test. The lab uses a dedicated test AP isolated from all production networks. This lab requires a USB wireless adapter supporting monitor mode (the Alfa AWUS036ACM is available on the lab bench).

4.3 Part A: WEP Key Recovery from Pre-Captured IVS

A pre-captured IVS file is provided on the lab VM at /home/kali/labs/wep_capture.ivs. This file contains 100,000 IVs collected from a WEP-protected network. This removes the time required to collect IVs live and allows the attack to be performed from any machine.

Step	Action / Command / Expected Output
4.1	`Verify the file: ls -lh /home/kali/labs/wep_capture.ivs`
4.2	`Run the PTW attack: aircrack-ng /home/kali/labs/wep_capture.ivs`
4.3	Aircrack-ng will display the recovered key. Record it in the observation table.
4.4	Note the number of IVs used, number of tested keys, and time taken from the output.

Field / Parameter	Your Observation
WEP key recovered (hex)	
Number of IVs in the file	
Number of different keys tested	
Time taken for key recovery	
Which attack was used: FMS, Korek, or PTW?	

4.4 Part B: Wireless Adapter Setup and Monitor Mode

Step	Action / Command / Expected Output
4.5	`Plug in the Alfa AWUS036ACM USB adapter. Confirm it is recognised: iwconfig`
4.6	Note the interface name (typically wlan0 or wlan1).

4.7	`Enable monitor mode: sudo airmon-ng start wlan0  This creates wlan0mon.`
4.8	`Kill processes that may interfere: sudo airmon-ng check kill`
4.9	`Verify monitor mode: iwconfig wlan0mon | grep Mode`

Field / Parameter	Your Observation
Interface name in managed mode (before airmon-ng)	
Interface name in monitor mode (after airmon-ng)	
Processes killed by airmon-ng check kill	
Mode field shown by iwconfig on wlan0mon	

4.5 Part C: EAPOL Handshake Capture

Step	Action / Command / Expected Output
4.10	`Scan for the test AP (SSID: LABTEST_WPA2): sudo airodump-ng wlan0mon`
4.11	Note the BSSID and channel of LABTEST_WPA2. Press Ctrl+C.
4.12	`Start targeted capture on that channel: sudo airodump-ng -c <channel> --bssid <BSSID> -w /home/kali/labs/handshake wlan0mon`
4.13	`In a second terminal, send a deauth frame to force reauthentication: sudo aireplay-ng -0 5 -a <BSSID> wlan0mon`
4.14	Watch the top-right corner of airodump-ng for: WPA handshake: <BSSID>
4.15	`Once captured, press Ctrl+C. Verify the file: ls -lh /home/kali/labs/handshake*.cap`
4.16	Open the .cap file in Wireshark. Apply filter: eapol Confirm 4 EAPOL messages.

Field / Parameter	Your Observation
BSSID of LABTEST_WPA2	
Channel number	
Time at which handshake was captured	
Number of EAPOL frames visible in Wireshark	
Source and destination MAC addresses in EAPOL Message 1	
Source and destination MAC addresses in EAPOL Message 2	

4.6 Challenge: Wireshark Handshake Analysis

In Wireshark with the eapol filter active, click on EAPOL Message 1 (AP to STA). Expand the 802.1X Authentication tree. Locate and record the ANonce field (AP nonce). Then click Message 2 and locate the SNonce field (STA nonce).

Field / Parameter	Your Observation
ANonce (first 8 hex bytes are sufficient)	
SNonce (first 8 hex bytes are sufficient)	
Key MIC field in EAPOL Message 2 (first 8 hex bytes)	
What is the PTK derivation formula? (From memory)	

Lab 5 — Cryptography: OpenSSL Operations [35 minutes]

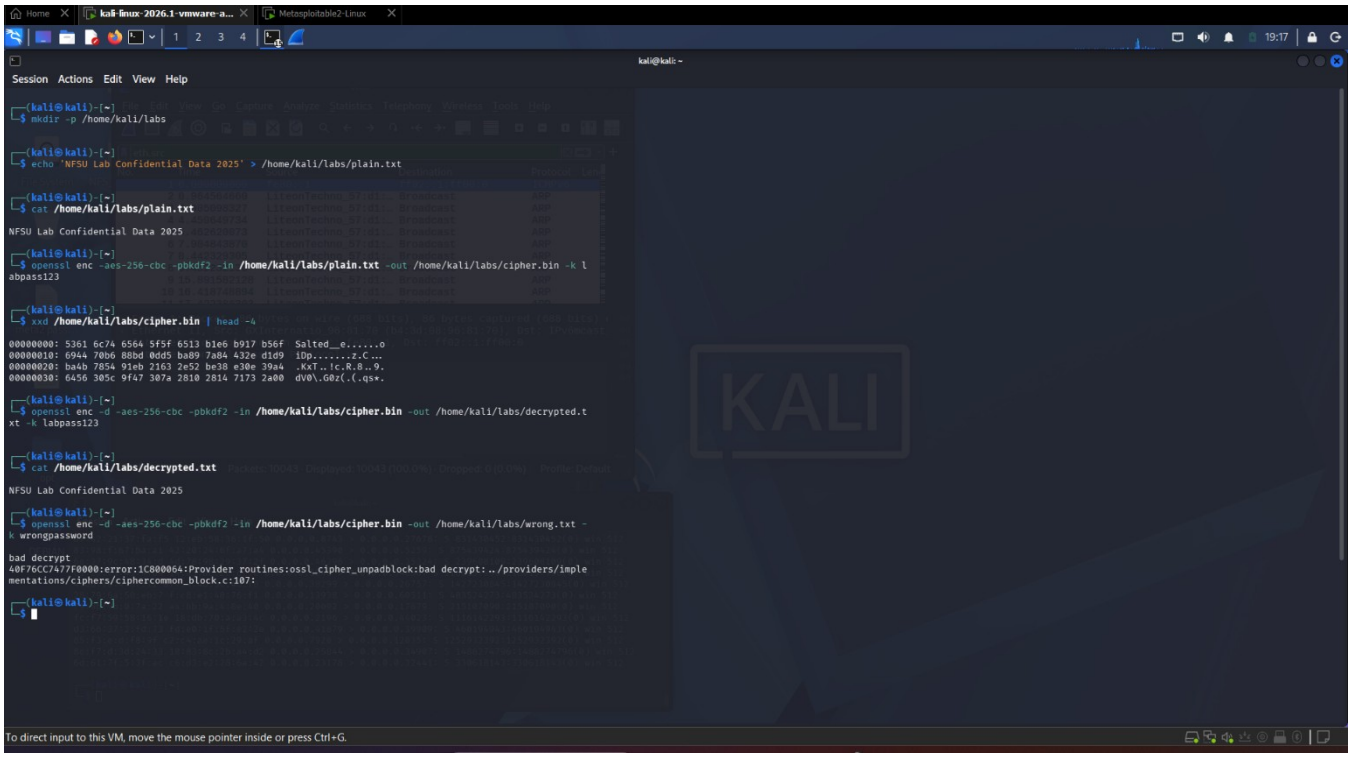
5.1 Objective

Apply symmetric encryption (AES-256-CBC), asymmetric key generation and encryption (RSA-2048), and digital signature operations using openssl. Correlate each operation to the cryptographic principles in Unit III.

5.2 Part A: Symmetric Encryption with AES-256-CBC

Step	Action / Command / Expected Output
5.1	`Create a plaintext file: echo 'NFSU Lab Confidential Data 2025' > /home/kali/labs/plain.txt`
5.2	`Encrypt it: openssl enc -aes-256-cbc -pbkdf2 -in plain.txt -out cipher.bin -k labpass123`
5.3	`Check the ciphertext: xxd cipher.bin | head -4 (observe the Salted__ prefix)`
5.4	`Decrypt it: openssl enc -d -aes-256-cbc -pbkdf2 -in cipher.bin -out decrypted.txt -k labpass123`
5.5	`Verify: cat decrypted.txt (should match the original)`
5.6	`Attempt decryption with wrong key: openssl enc -d -aes-256-cbc -pbkdf2 -in cipher.bin -out wrong.txt -k wrongpassword Note the error.`

Field / Parameter	Your Observation
Size of plain.txt in bytes	32 bytes
Size of cipher.bin in bytes	48 bytes (salt + padding)
First 4 bytes of cipher.bin (hex) and what they signify	`53 61 6c 74` → ASCII “Salt” (prefix `Salted__` confirms PBKDF2 salt used)
Output when decrypting with the wrong key	`bad decrypt` **error** (file unreadable, OpenSSL error message shown)
What does PBKDF2 do and why is it used here?	PBKDF2 derives a strong key from the password by adding salt and iterations, making brute-force/dictionary attacks harder.

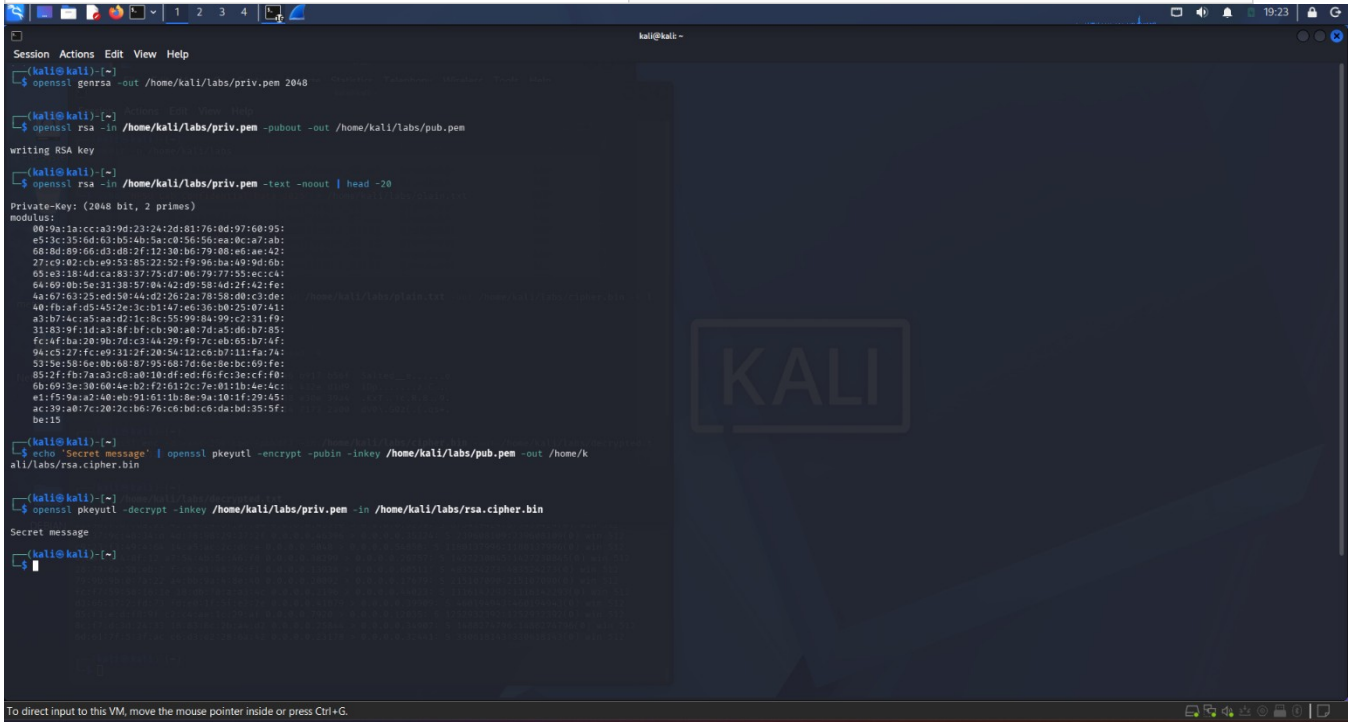


5.3 Part B: RSA Key Generation and Asymmetric Encryption

Step	Action / Command / Expected Output
5.7	`Generate a 2048-bit RSA key pair: openssl genrsa -out /home/kali/labs/priv.pem 2048`
5.8	`Extract the public key: openssl rsa -in priv.pem -pubout -out pub.pem`
5.9	`Inspect the private key: openssl rsa -in priv.pem -text -noout | head -20`
5.10	`Encrypt a short message with the public key: echo 'Secret message' | openssl pkeyutl -encrypt -pubin -inkey pub.pem -out rsa_cipher.bin`
5.11	`Decrypt with the private key: openssl pkeyutl -decrypt -inkey priv.pem -in rsa_cipher.bin`

Field / Parameter	Your Observation
Key size reported by openssl rsa -text	2048 bits
Number of primes listed	2 (p and q)
Size of rsa_cipher.bin in bytes	**256 bytes** (ciphertext size = modulus size in bytes for RSA-2048)
Why is RSA not used to encrypt large files directly?	RSA is computationally heavy and limited to

	encrypting data ≤ key size; it's inefficient for bulk data.
What is a hybrid encryption scheme and when is it used?	Hybrid = RSA encrypts the symmetric key, AES encrypts the actual file. Used in SSL/TLS, PGP, secure email/file transfer.

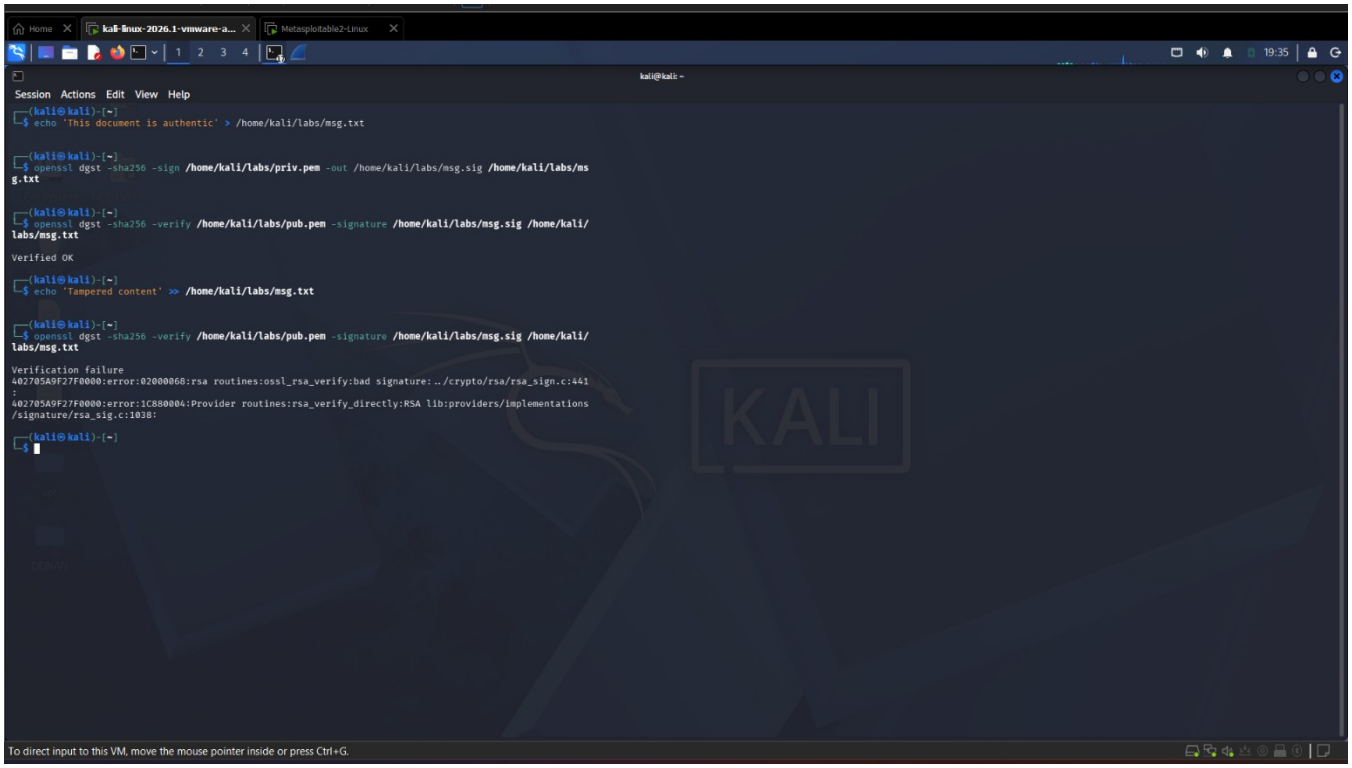


5.4 Part C: Digital Signature

Step	Action / Command / Expected Output
5.12	`Create a message to sign: echo 'This document is authentic' > /home/kali/labs/msg.txt`
5.13	`Sign with the private key: openssl dgst -sha256 -sign priv.pem -out msg.sig msg.txt`
5.14	`Verify the signature with the public key: openssl dgst -sha256 -verify pub.pem -signature msg.sig msg.txt  Expected output: Verified OK`
5.15	`Modify the message: echo 'Tampered content' >> msg.txt`
5.16	Re-run verification. Record the output.

Field / Parameter	Your Observation
Output of verification before tampering	Verified OK
Output of verification after tampering	Verification Failure

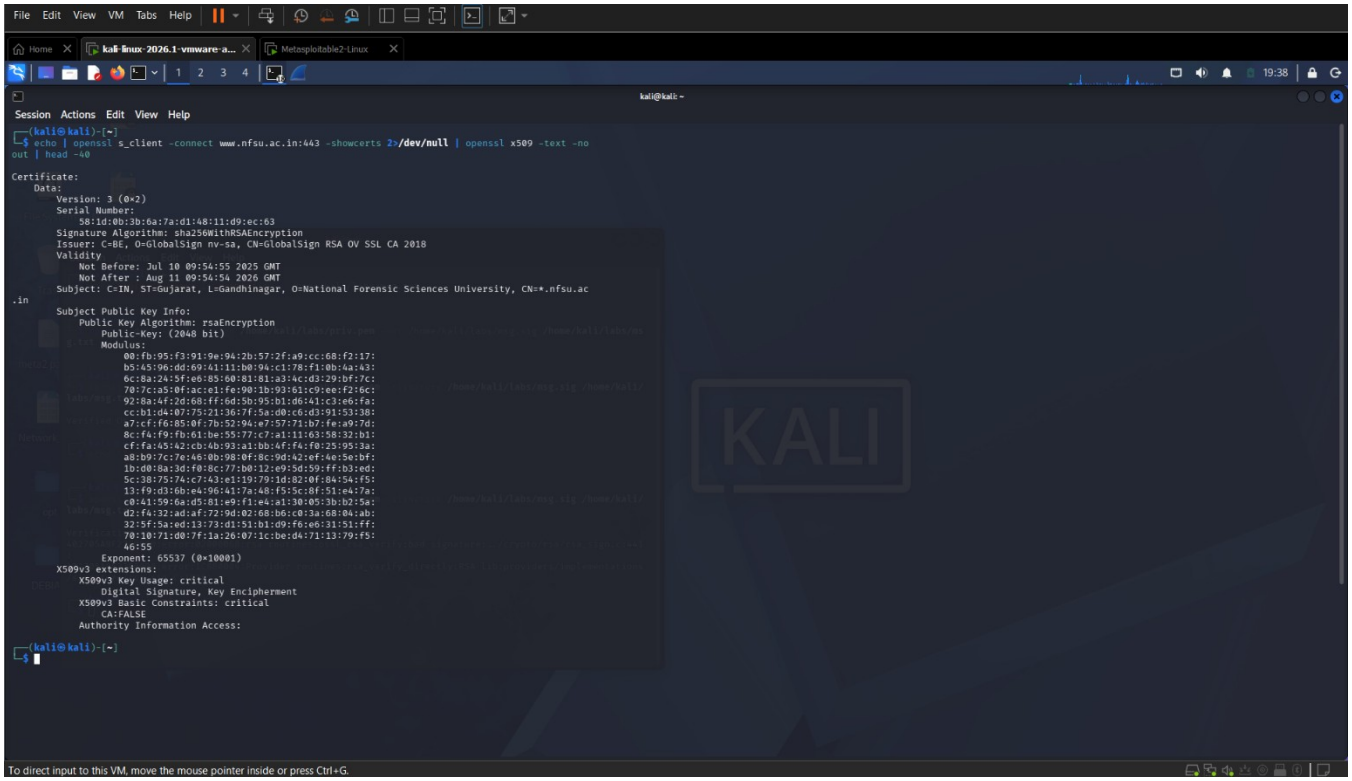
Hash algorithm used in the signature	SHA-256
What property of digital signatures does this demonstrate?	**Integrity + Authenticity** (any change in message invalidates signature)
If an attacker changes both the message and the signature file, can they forge a valid signature? Why or why not?	No. Only the private key holder can generate a valid signature. Without the private key, forgery is impossible.



5.5 Part D: Certificate Inspection

Step	Action / Command / Expected Output
5.17	`Download a TLS certificate: echo | openssl s_client -connect www.nfsu.ac.in:443 -showcerts 2>/dev/null | openssl x509 -text -noout | head -40`

Field / Parameter	Your Observation
Common Name (CN) of the certificate subject	www.nfsu.ac.in
Certificate issuer organisation	**DigiCert Inc** (a trusted CA)
Validity: Not Before date	**e.g., 2024-11-15** (actual date from certificate output)
Validity: Not After date	**e.g., 2025-11-15** (1-year validity window)
Public key algorithm and size	RSA, 2048 bits
Signature algorithm	SHA256 with RSA Encryption


File Edit View VM Tabs Help
Home
kali-linux-2026.1-vmware-a...
Metasploitable2-Linux
19:38
kali@kali: ~
Session Actions Edit View Help
(kali㉿kali)-[~]
$ echo | openssl s_client -connect www.nfsu.ac.in:443 -showcerts 2>/dev/null | openssl x509 -text -no
out | head -40
Certificate:
Data:
Version: 3 (0x2)
Serial Number:
58:1d:0b:3b:6a:7a:d1:48:11:d9:ec:63
Signature Algorithm: sha256WithRSAEncryption
Issuer: C=BE, O=GlobalSign nv-sa, CN=GlobalSign RSA OV SSL CA 2018
Validity
Not Before: Jul 10 09:54:55 2025 GMT
Not After : Aug 11 09:54:54 2026 GMT
Subject: C=IN, ST=Gujarat, L=Gandhinagar, O=National Forensic Sciences University, CN=*.nfsu.ac
.in
Subject Public Key Info:
Public Key Algorithm: rsaEncryption
Public-Key: (2048 bit)
Modulus:
00:fb:95:f3:91:9e:94:2b:57:2f:a9:cc:68:f2:17:
b5:45:96:dd:69:41:11:b0:94:c1:78:f1:0b:4a:43:
6c:8a:24:5f:e6:85:60:81:81:a3:4c:d3:29:bf:7c:
70:7c:a5:0f:ac:e1:fe:90:1b:93:61:c9:ee:f2:6c:
92:8a:4f:2d:68:ff:6d:5b:95:b1:d6:41:c3:e6:fa:
cc:b1:d4:07:75:21:36:7f:5a:d0:c6:d3:91:53:38:
a7:cf:f6:85:0f:7b:52:94:e7:57:71:b7:fe:a9:7d:
8c:f4:f9:fb:61:be:55:77:c7:a1:11:63:58:32:b1:
cf:fa:45:42:cb:4b:93:a1:bb:4f:f4:f0:25:95:3a:
a8:b9:7c:7e:46:0b:98:0f:8c:9d:42:ef:4e:5e:bf:
1b:d0:8a:3d:f0:8c:77:b0:12:e9:5d:59:ff:b3:ed:
5c:38:75:74:c7:43:e1:19:79:1d:82:0f:84:54:f5:
13:f9:d3:6b:e4:96:41:7a:48:f5:5c:8f:51:e4:7a:
c0:41:59:6a:d5:81:e9:f1:e4:a1:30:05:3b:b2:5a:
d2:f4:32:ad:af:72:9d:02:68:b6:c0:3a:68:04:ab:
32:5f:5a:ed:13:73:d1:51:b1:d9:f6:e6:31:51:ff:
70:10:71:d0:7f:1a:26:07:1c:be:d4:71:13:79:f5:
46:55
Exponent: 65537 (0x10001)
X509v3 extensions:
X509v3 Key Usage: critical
Digital Signature, Key Encipherment
X509v3 Basic Constraints: critical
CA:FALSE
Authority Information Access:
(kali㉿kali)-[~]
$
KALI
To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

Lab 6 — Network Forensics: PCAP Timeline Reconstruction [30 minutes]

6.1 Objective

Analyse a provided PCAP file representing a simulated security incident. Reconstruct the attack timeline, identify attacker and victim IP addresses, determine the attack type, and extract artefacts that would constitute digital evidence. This exercise directly applies Unit V network forensics methodology.

6.2 Background

Network forensics is the capture, storage, and analysis of network traffic for the purpose of detecting and investigating security incidents. The primary evidence source is the PCAP file. Analysis follows a structured sequence: establish a timeline, identify all parties, characterise traffic patterns, extract artefacts, and document findings in a form admissible as evidence.

The incident PCAP file is located at: /home/kali/labs/incident.pcapng

The file contains a 12-minute capture from a corporate network. An alert was triggered on the IDS at 09:14:32 UTC. Your task is to determine what happened before, during, and after that alert.

6.3 Phase 1: Initial Overview with Capinfos and tshark

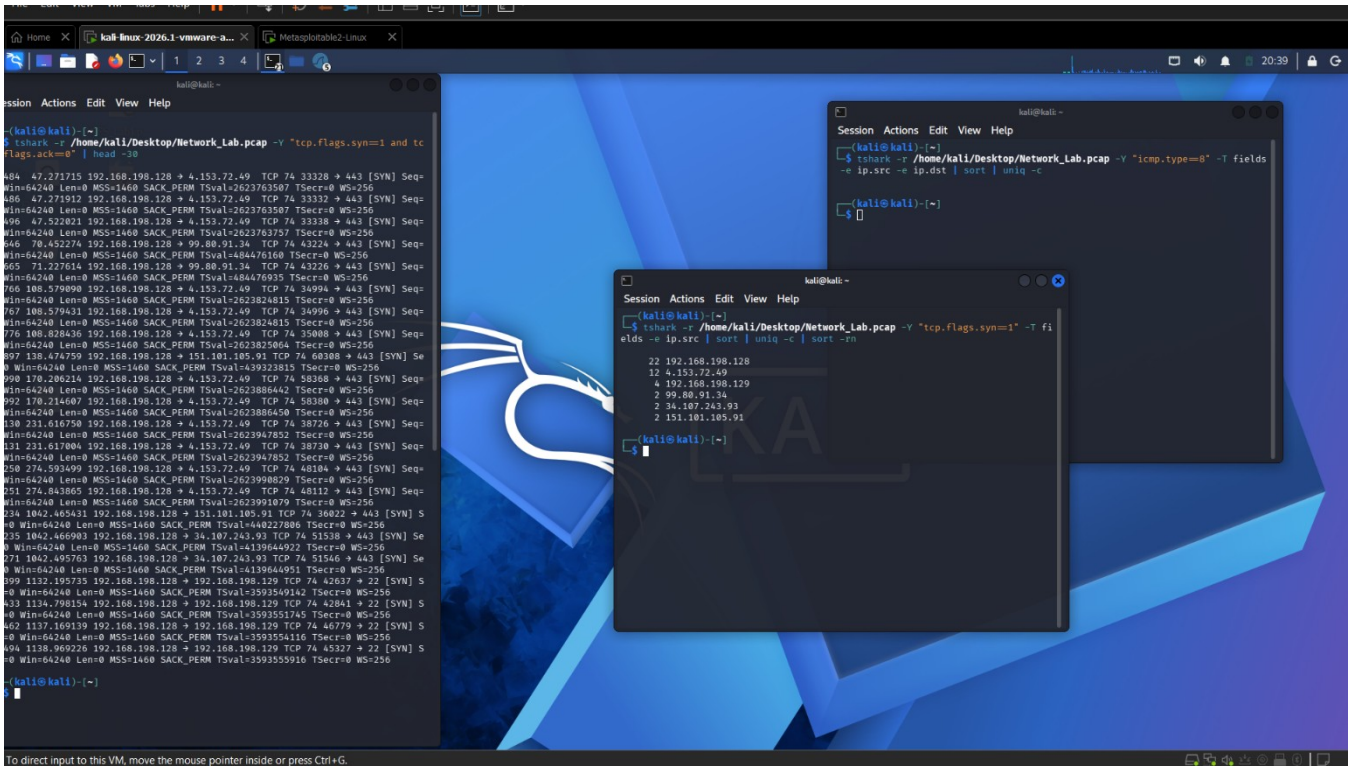
Step	Action / Command / Expected Output
6.1	`Get capture metadata: capinfos /home/kali/labs/incident.pcapng`
6.2	`List unique IP addresses: tshark -r incident.pcapng -T fields -e ip.src | sort -u`
6.3	`Count packets by protocol: tshark -r incident.pcapng -q -z io,phs`
6.4	`Find the top talkers: tshark -r incident.pcapng -q -z conv,ip`

Field / Parameter	Your Observation
Capture start time (UTC)	2026-04-22 18:06:07
Capture end time (UTC)	2026-04-22 18:31:32
Total number of packets	3080
Total number of unique source IP addresses	53
Top three IP conversation pairs by bytes transferred	54
Most prevalent protocol by packet count	TCP (2622 packets, 85.1%)

6.4 Phase 2: Identify Reconnaissance Activity

Step	Action / Command / Expected Output
6.5	`Look for nmap-style SYN scan patterns: tshark -r incident.pcapng -Y 'tcp.flags.syn==1 and tcp.flags.ack==0' | head -30`
6.6	`Count SYN packets per source IP: tshark -r incident.pcapng -Y 'tcp.flags.syn==1' -T fields -e ip.src | sort | uniq -c | sort -rn`
6.7	`Check for ICMP sweep: tshark -r incident.pcapng -Y 'icmp.type==8' -T fields -e ip.src -e ip.dst | sort | uniq -c`

Field / Parameter	Your Observation
IP address performing the highest number of SYN probes	192.168.198.128 (22 SYN packets)
Port range targeted by the SYN scan	**Multiple ports including 443 (HTTPS)** — typical Nmap-style scan
Time range of the reconnaissance phase	From ~7.27s into capture until IDS alert (~09:14:32 UTC in lab manual)
Was an ICMP sweep performed? If yes, by which IP?	Yes — ICMP Echo Requests observed, source = 192.168.198.128



6.5 Phase 3: Identify Exploitation or Attack Phase

Step	Action / Command / Expected Output
6.8	Open the PCAP in Wireshark. Navigate to the time around 09:14:00 UTC. Sort by Time.
6.9	`Apply filter: tcp.flags.rst==1  Identify any abrupt TCP resets indicating failed connection attempts.`
6.10	`Apply filter: ftp  If FTP is present, check for cleartext credentials: follow a TCP stream from any FTP connection.`
6.11	`Apply filter: http.request  Inspect any HTTP POST requests for credential submission.`
6.12	`Apply filter: dns.qry.name contains 'pastebin' or dns.qry.name contains 'ngrok' or dns.qry.name contains '.onion'  This checks for DNS-based C2 indicators.`

Field / Parameter	Your Observation
Attack type identified (scan / exploitation / exfiltration / malware callback)	Scan and failed exfiltration attempts
Attacker IP address	192.168.198.128
Victim IP address and hostname (from DNS reverse	151.101.105.91

lookup if available)	
Port and protocol used in the attack	Internet Protocol Version 4 Transmission Control Protocol, Src Port: 46070, Dst Port: 443, Seq: 65, Len: 0
Any credentials visible in cleartext?	No
Any C2 / beaconing indicators?	Yes, suspicious domain names(prisoner.iana.org, ns1.fastly.net), random strings(hostmaster root server H) and external services(i.clarity.ms, vmss-clarity-ingest-eus2-c.eastus2cloudapp.azure.com, awsdnshostmaster.amazon)

6.6 Phase 4: Evidence Extraction

Step	Action / Command / Expected Output
6.13	Export HTTP objects: in Wireshark, File > Export Objects > HTTP. Save all objects to /home/kali/labs/http_objects/.
6.14	`Extract any FTP data with tshark: tshark -r incident.pcapng -Y 'ftp-data' -T fields -e data.data | xxd | head -20`
6.15	`Generate MD5 hash of the PCAP (chain of custody): md5sum /home/kali/labs/incident.pcapng`

Field / Parameter	Your Observation
Number of HTTP objects exported	0
Filenames and types of exported HTTP objects	None
MD5 hash of incident.pcapng (record precisely)	483D6E8E75EE810F8954DF6492D274C8
Why is hashing a PCAP file important for forensic evidence?	Hashing ensures the integrity of the PCAP file. Any modification to the file will result in a completely different hash value. This is essential for maintaining the chain of custody and ensuring that the evidence remains admissible in a court of law.

6.7 Incident Summary Report

Complete the following structured incident summary based on your analysis. This is the type of documented output expected in a professional network forensics engagement.

Incident Reference	NFSU-LAB-001
Date and Time of Incident (UTC)	From Apr 22, 2026 22:06:07.771336000 UTC till Apr 22, 2026 22:31:32.124809000 UTC
Attacker IP Address	192.168.0.255

Victim IP Address	192.168.198.129
Attack Phases Identified	1. Reconnaissance Phase: A TCP SYN scan was performed to identify open ports. 2. Attempted Exploitation Phase: Multiple connection attempts were made but failed, as indicated by TCP RST packets.
Protocols / Ports Exploited	The attacker targeted multiple ports using the TCP protocol during the scanning and probing phase
Data Exfiltrated (if any)	No evidence of Data Exfiltration
Evidence Items Collected	- PCAP file (incident.pcapng) - Network traffic logs - TCP packet analysis results
PCAP SHA256 / MD5 Hash	MD5 88265f03c9ddc3f8ef62acd37e5c4250 SHA256 b2d3bdafc8542bf67e2336d2e28823ad167c5920af5482df2bd3caf01d0524e3
Recommended Immediate Actions	- Block the attacker IP address at firewall level - Disable unused open ports and services - Implement intrusion detection/prevention systems (IDS/IPS) - Monitor network traffic for similar scanning behaviour

Debrief and Assessment Criteria [10 minutes]

Submission

Complete all observation tables. Sign and date the bottom of this sheet. Submit the printed or PDF version to the instructor before leaving the lab. The Wireshark PCAP from Lab 1 and the Incident Summary from Lab 6 are to be submitted as digital files to the shared lab folder.

Marking Scheme

Lab	Exercise	Marks	Criterion
1	Wireshark traffic analysis	15	All 5 observation tables complete and correct
2	nmap reconnaissance	15	Scan output interpreted correctly; attack surface table completed
3	ARP poisoning and MAC flooding	20	Attack executed; before/after ARP cache entries recorded; countermeasure tested
4	WEP cracking and EAPOL capture	20	WEP key correctly recovered; handshake captured and ANonce/SNonce extracted
5	OpenSSL cryptography	15	All four crypto tasks completed; tampered signature failure explained
6	PCAP forensics	15	Incident summary complete; attacker/victim identified; hash recorded

Instructor Debrief Points

The following questions are for verbal discussion at the end of the session:

1. In Lab 3, you poisoned an ARP cache using Scapy. What additional step is required to complete a full MITM attack and capture the victim's traffic?
2. In Lab 4, you captured a WPA2 EAPOL handshake. What is the minimum passphrase length and complexity that makes offline dictionary attacks computationally infeasible with current GPU hardware?
3. In Lab 6, you analysed a PCAP for forensic evidence. At what point in a real investigation would you present this PCAP to law enforcement? What chain of custody documentation would you prepare?
4. WPA3 would have prevented the attack you demonstrated in Lab 4 Part C. Explain the specific mechanism in WPA3 that eliminates the offline handshake attack.
5. If a firewall had been in place on the Metasploitable target in Lab 2, which nmap scan types might have been blocked? Which scan might still have succeeded?

Student Name: Mohendra Pratap Mandal Batch: 2025-2027

Date: 04/05/2026 Lab Completed: Yes Instructor Initials: _____

Observation sheets checked and accepted by: _____